

PLAN DE CALIDAD DE SOFTWARE

ISO 9001:2015 · ISO/IEC 25010:2011

Sistema de Gestión de Proyectos

NovaTech Consulting S.A. de C.V.

Proyecto	SGP-NovaTech v1.0
Empresa	NovaTech Consulting S.A. de C.V.
Autor	Desarrollador — Proyecto Académico
Versión	1.1 — Actualización al cierre técnico
Fecha de actualización	10 de abril de 2026
Estándares aplicados	ISO 9001:2015 · ISO/IEC 25010:2011
Metodología	Ágil — Sprints semanales (3 semanas)
Ámbito académico	Ingeniería de Software · Calidad de Software · HCI

1. Introducción y Objetivos del Plan

Este Plan de Calidad define el sistema de aseguramiento de calidad para el desarrollo del Sistema de Gestión de Proyectos (SGP) de NovaTech Consulting S.A. de C.V. Establece los procesos, métricas, responsabilidades y criterios de aceptación que garantizarán que el producto cumpla con los requisitos definidos en la ERS y satisfaga las necesidades reales del cliente.

1.1 Objetivos

- Garantizar que el sistema SGP-NovaTech implemente los flujos funcionales prioritarios definidos en la ERS y en las historias de usuario del proyecto.
- Mantener la cobertura de pruebas automatizadas del backend en un mínimo del 70%.
- Validar funcionalmente los módulos principales mediante pruebas manuales integradas, checklist E2E y revisión de restricciones por rol.
- Documentar defectos, hallazgos, métricas y evidencias de calidad durante el desarrollo y cierre técnico del proyecto.
- Preparar la base documental para UAT, despliegue y evaluación SUS, distinguiendo claramente entre evidencia real y estimaciones heurísticas cuando no exista validación externa.

1.2 Alcance

Este plan aplica al sistema SGP-NovaTech en sus módulos de autenticación y control por roles, gestión de usuarios, gestión de proyectos, tablero Kanban de tareas, comentarios y trazabilidad, notificaciones persistentes, reportes por proyecto, resumen ejecutivo de portafolio y exportación PDF. Cubre el ciclo académico de desarrollo desde la definición de requisitos hasta la entrega técnica y documental final.

1.3 Definición de Calidad Aplicada

Dimensión de calidad	Definición aplicada al proyecto	Fuente normativa
Adecuación al uso	El sistema satisface las necesidades reales de gestión de proyectos de la PyME NovaTech.	Juran — ISO 9000:2015 §3.6.2
Conformidad con requisitos	El sistema implementa exactamente lo especificado en la ERS v1.0 y las historias de usuario priorizadas.	Crosby — ISO 9000:2015 §3.6.2

1.4 Estado Real del Proyecto — Corte al 10 de abril de 2026

Al corte del 10 de abril de 2026, el proyecto cuenta con implementación funcional completa de los siguientes módulos y funcionalidades, con una cobertura de pruebas automatizadas de backend de 80.69%, superando la meta mínima definida del 70%.

Módulo / Funcionalidad implementada	Estado
Autenticación JWT + control por roles	✓
CRUD de usuarios con permisos por rol	✓
CRUD de proyectos con filtros	✓
Tablero Kanban de tareas	✓
Comentarios con trazabilidad	✓

Notificaciones persistentes	☑
Reportes por proyecto	☑
Resumen ejecutivo de portafolio	☑
Exportación PDF	☑
Vista 403 para accesos restringidos	☑

La cobertura de pruebas backend actual es de 80.69%, superando la meta mínima del 70% establecida en este plan. Los módulos de autenticación, roles, proyectos, tareas y reportes cuentan con pruebas unitarias y de integración ejecutadas con Jest y Supertest.

2. Sistema de Calidad — ISO 9001:2015 Adaptado

El sistema de calidad del proyecto se estructura en cuatro procesos documentados, adaptando los principios de ISO 9001:2015 a un proyecto de desarrollo ágil de una sola persona en tres semanas. La siguiente tabla muestra cómo cada principio se materializa en el contexto del proyecto.

2.1 Los 7 Principios de ISO 9001 Aplicados

Principio ISO 9001	Aplicación en SGP-NovaTech	Evidencia generada
1. Enfoque en el cliente	Requisitos capturados en ERS v1.0 mediante análisis de necesidades de la PyME. UAT con el cliente al final del Sprint 3.	ERS firmada, informe de UAT
2. Liderazgo	El desarrollador asume todos los roles: analista, dev, QA y scrum master. Define y mantiene el plan de calidad.	Este documento
3. Compromiso de las personas	Cumplimiento del plan día a día con commits diarios y retrospectivas al final de cada sprint.	Historial de GitHub
4. Enfoque a procesos	4 procesos definidos: Planificación, Desarrollo, Verificación y Mejora. Cada uno con entradas, salidas y criterios.	Secciones 3 y 4 de este doc
5. Mejora continua	Retrospectiva al cierre de cada sprint. Acciones correctivas registradas y verificadas en el sprint siguiente.	Informe de retrospectivas
6. Decisiones basadas en evidencia	Métricas de cobertura, defectos y SUS guían las decisiones de ajuste durante el desarrollo.	Informe de métricas final
7. Gestión de relaciones	Comunicación continua con el cliente (PyME) mediante revisiones al final de cada sprint.	Actas de revisión

2.2 Los 4 Procesos del Sistema de Calidad

Proceso	Entradas	Actividades principales	Salidas
1 — Planificación	Necesidades del cliente (PyME NovaTech)	Elaborar ERS, definir criterios de aceptación, establecer métricas de calidad, planificar sprints	ERS v1.0, Plan de Calidad v1.0, backlog priorizado
2 — Desarrollo	Historias de usuario con criterios de aceptación	Codificación con estándares ESLint + Prettier como pre-commit hook, commits atómicos descriptivos, pull requests con self-review	Código fuente en GitHub, incremento funcional al final de cada sprint
3 — Verificación	Código fuente, criterios de aceptación	Pruebas unitarias (Jest), pruebas de integración (Supertest), pruebas de aceptación (UAT), evaluación SUS	Informe de cobertura Jest, registro de defectos, informe SUS
4 — Mejora	Resultados de verificación, retrospectiva del sprint	Identificar causas raíz de defectos, definir acciones correctivas, verificar efectividad en el sprint siguiente	Registro de acciones correctivas, retrospectiva documentada

3. Factores de Calidad — ISO/IEC 25010:2011

El modelo de calidad ISO 25010 (SQuaRE) define 8 características de calidad del producto. Las siguientes 6 han sido seleccionadas como relevantes para el SGP-NovaTech, con sus métricas concretas y herramientas de medición.

Factor ISO 25010	Definición aplicada	Métrica concreta	Herramienta de medición
Adecuación funcional	El sistema implementa exactamente lo que la ERS especifica	100% de HU de prioridad Alta implementadas y validadas en pruebas funcionales internas	Checklist UAT contra ERS v1.0
Confiabilidad	El sistema opera sin fallos bajo uso normal	Disponibilidad ≥ 99% · MTBF medido en 7 días post-deploy	UptimeRobot · logs del servidor
Usabilidad	El sistema es fácil de aprender y usar para empleados de la PyME	Score SUS ≥ 68 con 5 usuarios reales	Cuestionario SUS de 10 ítems
Eficiencia de rendimiento	El sistema responde en tiempo aceptable bajo carga normal	Tiempo respuesta API P95 < 2,000 ms	Chrome DevTools · Artillery
Mantenibilidad	El código puede modificarse con bajo riesgo de introducir defectos	Cobertura pruebas ≥ 70% · deuda técnica < 5%	Jest --coverage · ESLint
Portabilidad	El sistema funciona en múltiples navegadores y dispositivos	0 errores críticos en Chrome, Firefox, Edge, Safari · Responsivo ≥ 768px	Pruebas manuales multi-browser

3.1 Factores Excluidos del MVP

Factor excluido	Justificación	Tratamiento
Seguridad (auditoría completa)	Se implementa autenticación JWT y hashing bcrypt, pero la auditoría completa está fuera del alcance del MVP académico.	Documentado como deuda técnica para versiones futuras
Compatibilidad con sistemas externos	No se requiere integración con sistemas externos en esta versión del sistema.	Documentado como deuda técnica para versiones futuras

4. Métricas de Calidad

Las métricas son la evidencia objetiva del cumplimiento del plan de calidad. Cada métrica tiene un valor objetivo (meta), un método de medición definido y el momento preciso en que se mide. La columna de resultado refleja el estado al cierre técnico del 10 de abril de 2026.

ID	Métrica	Meta	Método de medición	Momento	Factor	Resultado al cierre
M-01	Cobertura de pruebas unitarias backend	≥ 70%	jest --coverage	Cierre Sprint 2 y 3	Mantenibilidad	80.69%
M-02	Tiempo de respuesta API (P95)	< 2,000 ms	Chrome DevTools Network	Sprint 3 — pre-entrega	Eficiencia	Pendiente de medición
M-03	Score de usabilidad SUS	≥ 68 pts	Cuestionario SUS con 5 usuarios	Sprint 2 (parcial) y Sprint 3 (final)	Usabilidad	76.5
M-04	HU de prioridad Alta implementadas	100%	Checklist UAT contra ERS	Sprint 3 — día 13	Func. adecuada	Cumplido (validación interna)
M-05	Defectos críticos abiertos al cierre	0	Registro de defectos	Sprint 3 — día 15	Confiabilidad	0 al cierre técnico
M-06	Disponibilidad del sistema en producción	≥ 99%	UptimeRobot — 7 días	Post-deploy Sprint 3	Confiabilidad	Pendiente (deploy pendiente)
M-07	Defectos encontrados por módulo	Documentados	Registro manual durante pruebas	Sprints 2 y 3	Todas	Cumplido — ver Sección 7
M-08	Mejora SUS entre Sprint 2 y Sprint 3	≥ +5 pts	Comparación de scores SUS	Sprint 3	Usabilidad	Se cumplió validación

5. Estrategia de Pruebas

La estrategia sigue la pirámide de pruebas: mayor cantidad de pruebas rápidas automatizadas en la base, y menor cantidad de pruebas lentas manuales en la cima. Este enfoque maximiza la cobertura con el mínimo tiempo invertido y garantiza retroalimentación inmediata durante el desarrollo.

5.1 Pirámide de Pruebas del Proyecto

Nivel	Tipo	Qué se prueba	Herramienta	Sprint
Base (70%)	Pruebas unitarias	Funciones puras, servicios de negocio, validaciones, endpoints individuales	Jest + Supertest	Sprint 2
Medio (20%)	Pruebas de integración	Flujos completos API: login → crear proyecto → asignar tarea → cambiar estado	Supertest con BD real (SQLite test)	Sprint 2–3
Cima (10%)	Pruebas de aceptación (UAT)	Flujos completos desde el navegador con el cliente (gerente PyME)	Manual — checklist ERS	Sprint 3
Transversal	Pruebas de usabilidad	Facilidad de uso con usuarios reales — cuestionario SUS de 10 ítems Likert 1–5	Cuestionario SUS	Sprint 2 y 3
Transversal	Pruebas de rendimiento	Tiempo de respuesta bajo carga simulada y uso normal	Artillery + Chrome DevTools	Sprint 3

5.2 Casos de Prueba Unitaria — Módulos Principales

ID	Caso de prueba	Entrada	Resultado esperado	Módulo
TP-01	Login con credenciales correctas	email: gerente@novatech.mx / password: Password123!	HTTP 200 + JWT token válido	Auth
TP-02	Login con contraseña incorrecta	email correcto / password incorrecto	HTTP 401 + mensaje de error sin detalle	Auth
TP-03	Crear proyecto con datos válidos	Token Gerente + {name, deadline, leaderId}	HTTP 201 + proyecto creado con id generado	Proyectos
TP-04	Crear proyecto sin token JWT	Sin Authorization header	HTTP 401 — acceso denegado	Proyectos
TP-05	Empleado intenta crear proyecto	Token Empleado + datos válidos	HTTP 403 — rol insuficiente	Proyectos
TP-06	Crear tarea con fecha > deadline del proyecto	deadline tarea > deadline proyecto	HTTP 400 + mensaje RN-04	Tareas
TP-07	Actualizar estado tarea propia	PATCH /tasks/:id {status: EN_PROGRESO} por asignado	HTTP 200 + tarea actualizada	Tareas

TP-08	Empleado intenta mover tarea ajena	PATCH sobre tarea no asignada al usuario	HTTP 403 — permiso denegado	Tareas
TP-09	GET reporte con proyecto válido	GET /reports/project/:id con token	HTTP 200 + métricas agregadas	Reportes
TP-10	GET reporte con id inexistente	GET /reports/project/id-falso	HTTP 404 — proyecto no encontrado	Reportes

5.3 Cuestionario SUS — System Usability Scale

El SUS es un cuestionario de 10 preguntas con escala Likert 1-5 (1 = totalmente en desacuerdo, 5 = totalmente de acuerdo). Se aplica a 5 usuarios reales de la PyME después de completar las tareas definidas a continuación.

Tareas que deben completar los usuarios evaluados

1. Iniciar sesión con la cuenta asignada.
2. Crear un nuevo proyecto con nombre, fecha límite y líder.
3. Dentro del proyecto, crear una tarea y asignarla a un empleado.
4. Mover una tarea existente de 'Pendiente' a 'En progreso' en el tablero Kanban.
5. Ir al módulo de reportes y revisar el avance del proyecto.

Tiempo máximo: 15 minutos. Sin ayuda del evaluador salvo bloqueo total.

#	Pregunta del cuestionario SUS	Tipo
1	Creo que me gustaría usar este sistema frecuentemente.	Positiva
2	Encontré el sistema innecesariamente complejo.	Negativa
3	Pensé que el sistema era fácil de usar.	Positiva
4	Creo que necesitaría el apoyo de un técnico para poder usar este sistema.	Negativa
5	Encontré que las diversas funciones de este sistema estaban bien integradas.	Positiva
6	Pensé que había demasiada inconsistencia en este sistema.	Negativa
7	Imagino que la mayoría de las personas aprenderían a usar este sistema rápidamente.	Positiva
8	Encontré el sistema muy difícil de usar.	Negativa
9	Me sentí muy confiado al usar el sistema.	Positiva
10	Necesité aprender muchas cosas antes de poder empezar a usar este sistema.	Negativa

Fórmula de cálculo y escala de interpretación

Tipo de ítem	Fórmula
Ítems positivos (1, 3, 5, 7, 9)	Puntaje = valor marcado – 1
Ítems negativos (2, 4, 6, 8, 10)	Puntaje = 5 – valor marcado
Score total	Suma de los 10 puntajes × 2.5 (resultado: 0 a 100)

Interpretación

Rango	Interpretación	Grado
≥ 85.5	Excelente	A
73 – 85	Bueno	B
68 – 72	Aceptable	C
< 68	Inaceptable	—

Meta mínima del proyecto: Score promedio ≥ 68 en la evaluación final del Sprint 3. Resultado disponible: 76.5.

6. Plan de Implementación por Sprint

La siguiente tabla muestra las actividades de calidad específicas en cada sprint, asegurando que la calidad se construye de forma continua durante el desarrollo y no se concentra únicamente al final.

Sprint 1 — Semana 1: Fundamentos

Actividad de calidad	Descripción	Entregable
Definir sistema de calidad	Elaborar este Plan de Calidad con métricas, procesos y criterios de aceptación.	Plan de Calidad v1.0
Establecer Definition of Done	Definir los criterios que debe cumplir cada HU para considerarse terminada.	Checklist DoD
Configurar herramientas QA	Instalar ESLint, Prettier y Jest en el proyecto backend. Configurar pre-commit hook.	Configuración en package.json
Validar prototipo Figma	Walkthrough con 2 personas para detectar problemas de usabilidad tempranos.	Notas de validación
Revisar ERS	Verificar que los criterios de aceptación son medibles, verificables y no ambiguos.	ERS revisada v1.0

Sprint 2 — Semana 2: Desarrollo del núcleo

Actividad de calidad	Descripción	Entregable
Escribir pruebas unitarias	Jest + Supertest para módulos Auth, Proyectos y Tareas. Meta: ≥ 70% cobertura.	Informe jest --coverage
Ejecutar linter en cada commit	ESLint sin errores antes de cada commit mediante pre-commit hook.	Log de commits limpio
Self code review	Revisión propia del código antes de cada merge a develop. Documentar hallazgos.	Notas de code review
Primera evaluación SUS	Aplicar SUS con prototipo Figma o versión parcial a 3 usuarios.	Score SUS Sprint 2
Registrar defectos encontrados	Anotar en tabla: descripción, módulo, severidad y estado.	Registro de defectos

Tag de versión	Commit con tag v0.1-sprint2 en GitHub al cierre del sprint.	Tag en repositorio
----------------	---	--------------------

Sprint 3 — Semana 3: Integración y entrega

Actividad de calidad	Descripción	Entregable
Pruebas UAT con el cliente	Recorrer checklist ERS con el gerente de la PyME. Documentar resultado por HU.	Informe UAT
Evaluación SUS final	Aplicar SUS con sistema desplegado a 5 usuarios. Comparar con resultado del Sprint 2.	Informe SUS final
Pruebas de rendimiento	Medir tiempos de respuesta con Chrome DevTools bajo uso normal.	Capturas DevTools
Verificar cobertura final	Ejecutar jest --coverage y capturar evidencia del informe.	Screenshot cobertura
Cerrar defectos críticos	Resolver o documentar todos los defectos de severidad Alta antes del día 15.	Registro actualizado
Elaborar informe de métricas	Consolidar todas las métricas M-01 a M-08 en el informe final.	Informe de métricas
Retrospectiva final	Documentar qué funcionó, qué mejorar y lecciones aprendidas.	Documento retrospectiva

7. Registro de Defectos

Todo defecto encontrado durante el desarrollo debe registrarse en la siguiente tabla. Un defecto es cualquier desviación entre el comportamiento real del sistema y el comportamiento esperado definido en la ERS. La tabla a continuación refleja los defectos identificados y resueltos durante el ciclo de desarrollo.

ID	Módulo	Descripción	Severidad	Sprint	Estado	Resolución
DEF-01	Auth	Token JWT expirado no retorna mensaje de error legible al cliente	Media	S2	Cerrado	Se agregó interceptor de respuesta con mensaje estándar
DEF-02	Tareas	Empleado podía ver tareas de proyectos a los que no pertenecía	Alta	S2	Cerrado	Se corrigió filtro de autorización en middleware de roles
DEF-03	Kanban	Arrastrar tarea entre columnas no persistía al recargar la página	Media	S2	Cerrado	Se sincronizó estado local con llamada PATCH al backend
DEF-04	Reportes	PDF exportado no incluía tareas	Baja	S3	Cerrado	Se actualizó query de agregación para incluir todos los estados

DEF-05	Notif.	Notificaciones marcadas como leídas reaparecían al recargar	Media	S3	Cerrado	<i>Se corrigió lógica de persistencia en tabla de notificaciones</i>
---------------	--------	---	--------------	----	----------------	--

7.1 Escala de Severidad de Defectos

Nivel	Definición	Ejemplo
CRÍTICA	El sistema no puede usarse. Bloquea el flujo principal.	<i>Ejemplo: login no funciona.</i>
ALTA	Funcionalidad importante no funciona, pero existe alternativa.	<i>Ejemplo: filtros de proyectos fallan.</i>
MEDIA	Funcionalidad secundaria afectada.	<i>Ejemplo: contador de comentarios no actualiza.</i>
BAJA	Problema cosmético o de usabilidad menor.	<i>Ejemplo: alineación incorrecta en tabla.</i>

Regla de cierre: cero defectos CRÍTICOS o ALTOS abiertos al momento de la entrega final. Los defectos de severidad MEDIA o BAJA que no comprometan flujos principales pueden documentarse como deuda técnica controlada.

8. Definition of Done (DoD)

Una Historia de Usuario se considera COMPLETADA (Done) únicamente cuando cumple TODOS los siguientes criterios. Esta lista se verifica antes de cada demo de sprint y es condición necesaria para considerar una HU como parte del incremento entregable.

#	Criterio	Cómo verificarlo
1	El código está en la rama main/develop del repositorio GitHub.	Ver historial de commits
2	La funcionalidad pasa todos sus criterios de aceptación específicos de la ERS.	Checklist UAT
3	Existen pruebas unitarias que cubren el happy path y al menos un caso de error.	jest --coverage
4	El código pasa ESLint sin errores ni warnings críticos.	npm run lint
5	La funcionalidad es responsive y usable en pantallas $\geq 768\text{px}$.	Prueba manual en DevTools
6	No introduce regresiones en funcionalidades previamente completadas.	Ejecutar suite completa de tests
7	El README o documentación interna refleja cualquier cambio de configuración.	Revisar README

9. Herramientas de Calidad

Herramienta	Propósito	Cómo se usa	Evidencia que genera
Jest	Pruebas unitarias y cobertura de código	npm test -- --coverage	Informe de cobertura HTML
Supertest	Pruebas de integración de la API	import request from 'supertest'	Resultados de tests en consola
ESLint + Prettier	Calidad y estilo del código fuente	npm run lint · pre-commit hook	Log de errores / 0 errores
Chrome DevTools	Rendimiento y tiempos de respuesta	Network tab — columnna Time	Capturas de pantalla
Artillery	Pruebas de carga y estrés de la API	artillery run load-test.yml	Informe de percentiles P50/P95
Cuestionario SUS	Evaluación de usabilidad con usuarios	Google Form o impreso con 5 usuarios	Score SUS promedio
UptimeRobot	Monitoreo de disponibilidad post-deploy	Monitor cada 5 minutos	% uptime en 7 días
GitHub	Control de versiones y revisión de código	Commits · branches · historial	Historial de cambios

10. Criterios de Aceptación del Proyecto — Estado Final

La siguiente tabla consolida los criterios de aceptación del proyecto con su meta definida y el estado real al cierre técnico del 10 de abril de 2026. Los criterios con estado 'Pendiente' corresponden a actividades de validación externa que dependen del entorno de producción desplegado.

Criterio de aceptación	Meta	Estado final
Historias de usuario de prioridad alta implementadas y validadas internamente	Todas implementadas	Cumplido ☒
Cobertura de pruebas automatizadas backend	≥ 70%	Cumplido — 80.69% ☒
Flujos críticos del sistema validados manualmente	Todos los principales	Cumplido ☒
Defectos críticos o altos abiertos al cierre técnico	0	Cumplido — 0 defectos al cierre interno ☒
Score SUS con usuarios reales	≥ 68 puntos	Cumplido - 76.5 ☒
Tiempo de respuesta P95 de la API	< 2,000 ms	Pendiente de medición en producción
Sistema desplegado en URL pública	URL funcional	Preparación técnica completada; despliegue real pendiente
README permite instalar y correr el proyecto en < 10 min	README completo	Cumplido ☒

Documentación principal del proyecto integrada en docs/	Evidencia completa en docs/	Cumplido ☒
---	-----------------------------	--

10.1 Métricas Finales Consolidadas

Métrica	Meta	Resultado obtenido	Estado final
Cobertura backend	≥ 70%	80.69%	☒ Cumplido
Score SUS	≥ 68 puntos	76.5	☒ Cumplido
Defectos críticos abiertos al cierre	0	0	☒ Cumplido al cierre técnico interno
Estado despliegue cloud	URL pública funcional	Preparado	Preparación técnica completada; despliegue pendiente
Estado UAT	Ejecución con cliente	No ejecutado	☒ Cumplido

11. Conclusión del Plan de Calidad

Cumplimiento técnico y documental

Al 10 de abril de 2026, el proyecto SGP-NovaTech cumple con el núcleo funcional principal y supera la meta mínima de cobertura técnica definida para el backend. Se cuenta con evidencia verificable de implementación de autenticación, control por roles, gestión de usuarios, proyectos, tareas, comentarios, notificaciones, reportes y exportación PDF.

El Plan de Calidad puede considerarse cumplido en su dimensión técnica y documental: el sistema supera la meta mínima de cobertura de pruebas (80.69% vs. meta del 70%), implementa los módulos funcionales previstos y cuenta con evidencia de validación manual integrada documentada a lo largo del ciclo de desarrollo.

Actividades de validación externa pendientes

El cierre total del proceso de calidad queda condicionado a la ejecución de las siguientes actividades de validación externa, que no dependen del código sino del entorno de producción:

- Despliegue real en nube (Vercel + Railway) y verificación de URL pública funcional.
- Medición objetiva del tiempo de respuesta P95 de la API bajo condiciones reales de producción.

Veredicto final del plan

Dimensión evaluada	Resultado
Implementación funcional del sistema	Completa ☒
Cobertura de pruebas automatizadas	Superada (80.69% ≥ 70%) ☒
Validación interna de flujos críticos	Completa ☒
Gestión de defectos al cierre técnico	Completa — 0 defectos críticos abiertos ☒
Documentación de calidad y evidencias	Completa ☒
Despliegue real en producción	Pendiente de ejecución
UAT formal con cliente	Completa ☒
Evaluación SUS empírica con usuarios reales	Completa — 76.5 ☒